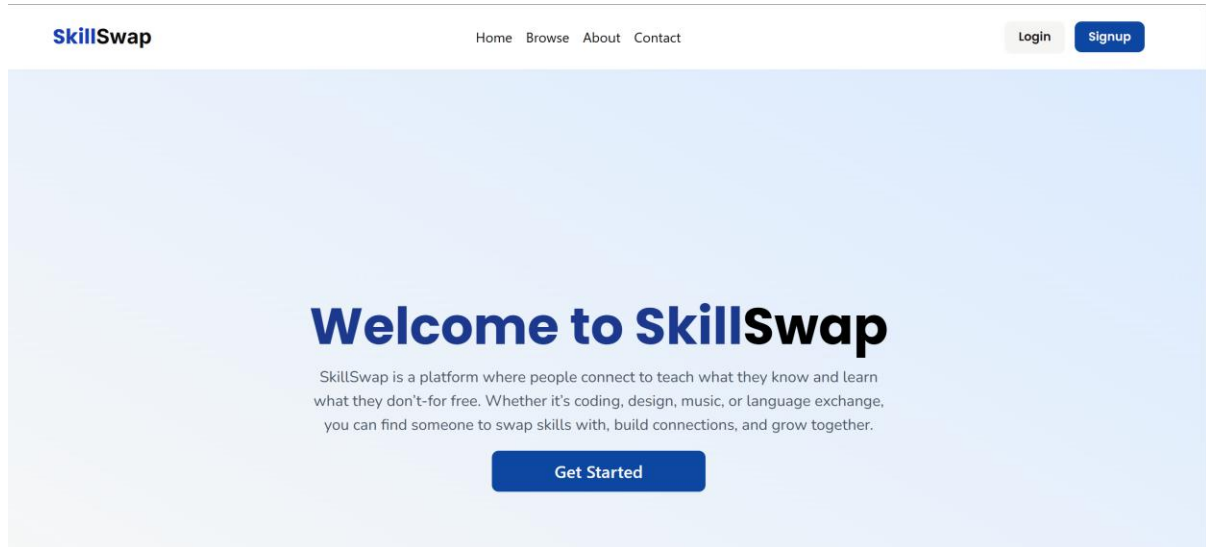


SkillSwap — Skill Exchange Platform

Developed by : Adarsh Srikakolapu (24m0317)

github link : [Gitlink](#)

Guide Under : *prof. Deepak Singh*



Abstract

SkillSwap is an end-to-end MERN (MongoDB, Express, React, Node) application that enables users to exchange micro-skills instead of monetary transactions. This report presents an implementation blueprint designed for a beginner-to-intermediate full-stack developer: functional requirements, architecture, data model, matchmaking algorithm, real-time chat integration, testing strategy, deployment (Vercel/Render), CI/CD pipeline, and scalability recommendations. The goal is to provide a production-grade deliverable that is interview-ready and demonstrates system design, engineering tradeoffs, and deployment know-how.

Keywords: MERN, matchmaking, Socket.io, MongoDB Atlas, CI/CD, Vercel, Render, JWT, Redis, scalability.

1. Project Overview & Objectives

Problem statement: Many learners and professionals seek quick, practical skill exchanges (e.g., interview prep for a designer in exchange for a front-end lesson). Existing marketplaces are money-centric; SkillSwap focuses on peer exchange and low friction matchmaking.

Objectives:

- Build a secure, responsive MERN web app with real-time chat.
- Implement a deterministic, explainable matchmaking algorithm.

- Ship a CI/CD pipeline and deploy frontend/backend to Vercel/Render with MongoDB Atlas.
- Demonstrate testing, monitoring, and scalability best practices.

Success metrics to show recruiters: Match conversion rate, successful swaps completed, average time-to-match, number of daily active users (DAU), 95th/99th percentile API latency.

2. High-Level Architecture

flowchart LR

Client[React + Tailwind] -->|REST API| API[Express.js / Node]

Client -->|Socket.io| Socket[Socket.io Server]

API --> Mongo[MongoDB Atlas]

API --> Auth[JWT / bcrypt]

Socket --> Redis[Redis Adapter (for scale)]

API --> Redis

CI[GitHub Actions] --> DeployFE[Vercel]

CI --> DeployBE[Render]

Architecture notes:

- Frontend (React) communicates via REST for CRUD and via Socket.io for chat/notifications.
- Backend (Express) exposes auth, user, matchmaking, swap, and message routes.
- MongoDB Atlas stores users, requests, matches and messages; add indexes for tags and timestamps.
- Redis used as a session/cache and Socket.io adapter when scaling horizontally.

Alternate ASCII flow: React <-> Express <-> MongoDB and React <-> Socket.io <-> Node (Redis adapter).

Learn something new for free!

Get a new friend that will manage to help you achieve your goal!



Skill Marketplace

Browse a diverse catalog of skills and knowledge areas, from cooking and coding to photography and foreign languages. Find what piques your interest.



New Interface

Connect with people who share your passion for learning and teaching. Propose skill swaps, schedule sessions, and get ready to trade knowledge.



SkillSwap

Is all about the joy of learning and sharing. No matter your skill level, you can be a valuable contributor and learner in this vibrant community. Discover new passions, connect with like-minded individuals, and watch your skills flourish. Download SkillSwap today and embark on a journey of skill and knowledge exchange like never before.

3. Core Features & Flowchart

Features:

1. User registration & profile (skills offered / skills wanted).
2. Search & filter by skill tags and availability.
3. Smart matchmaking (mutual skill overlap scoring).
4. Swap requests (accept/decline/state machine).
5. Real-time chat for matched pairs with message persistence.
6. Reputation points & basic gamification.

Matchmaking flow (sequence):

sequenceDiagram

participant U as User

participant C as Client

participant A as API

participant M as MatchingService

U->>C: Submit "Offer: Resume Writing" "Want: Python"

C->>A: POST /api/swaps

A->>M: evaluate(request)

M->>A: match (if mutual)

A->>Mongo: create Swap + Link users

A->>C: 200 OK + notification

A->>Socket: emit("match", data)

4. Data Model (Key Schemas)

User (collection: users)

```
{
  "_id": "ObjectId",
  "name": "String",
  "email": "String",
  "passwordHash": "String",
  "skillsOffered": ["strings"],
  "skillsWanted": ["strings"],
  "reputation": 0,
  "createdAt": "Date"
}
```

SwapRequest (collection: swaps)

```
{
  "_id": "ObjectId",
  "fromUser": "ObjectId",
  "toUser": "ObjectId",
  "offeredSkill": "String",
  "wantedSkill": "String",
  "status": "pending|accepted|declined|completed",
  "createdAt": "Date"
}
```

Message (collection: messages)

```
{
  "_id": "ObjectId",
  "swapId": "ObjectId",
  "sender": "ObjectId",
  "text": "String",
  "createdAt": "Date"
}
```

}

Indexes: create index on skillsOffered, skillsWanted, createdAt and compound indexes used in matchmaking queries.

5. API Endpoints (Minimal & Recruiter-friendly)

- POST /api/auth/register — register user (bcrypt + email validation)
 - POST /api/auth/login — returns JWT
 - GET /api/users/:id — profile
 - GET /api/search?skill=python — tag search with pagination
 - POST /api/swaps — create swap request (calls matching)
 - PUT /api/swaps/:id — update status (accept/decline/complete)
 - GET /api/swaps — list user swaps
 - WS /socket.io — events: connect, join_swap_<id>, message, match_notification
-

6. Matchmaking Algorithm (Concise + Explainable)

Design: simple, deterministic scoring using tag overlap and reputation.

Score formula (example):

$\text{score} = \text{overlapScore} * (1 + \min(0.2, \text{reputation}/100))$

where $\text{overlapScore} = \text{numberOfMatchingPairs} / \text{maxPossiblePairs}$.

Pseudocode:

```
function findMatches(request){
  candidates = findUsersBySkill(request.wantedSkill)
  for c in candidates:
    if request.offeredSkill in c.skillsWanted:
      score = computeScore(request, c)
  return topN by score
}
```

Complexity: typical query uses indexed lookups ($O(\log n)$ for DB index + aggregation), suitable for small/medium user base.

7. Real-time Chat Integration

Socket.io pattern:

- Room name: swap_<swapId>
- Events: join, leave, message:send, message:receive, typing
- Persist messages in messages collection to enable history on reconnect.

Scaling note: when running multiple Node instances, use socket.io-redis adapter to broadcast messages across instances.

8. Authentication, Security & Best Practices

- Use **bcrypt** for hashing (salt rounds 10–12).
 - Issue short-lived **JWT access tokens** + refresh tokens stored securely (httpOnly cookie).
 - Validate inputs with JOI / celebrate and use helmet, rate-limiting and CORS policy.
 - Sanitize stored messages to avoid XSS on render.
 - Use HTTPS in production; enable HSTS and TLS via hosting provider.
-

9. Testing & Quality Assurance

Unit tests: Jest for utility and algorithm tests (matchmaking logic).

Integration tests: Supertest for API endpoints.

E2E tests: Cypress to validate flows (signup → create swap → accept → chat).

Load testing: k6 or Artillery to simulate concurrent swaps and chat messages.

CI: run lint → test → build on push/PR (GitHub Actions).

10. Deployment & CI/CD (Step-by-Step)

Backend (Render)

1. Connect GitHub repo to Render as a Web Service.
2. Set environment variables: MONGO_URI, JWT_SECRET, REDIS_URL, NODE_ENV=production.
3. Build command: npm ci && npm run build
4. Start command: npm run start

Frontend (Vercel)

1. Link GitHub repo to Vercel.
2. Build command: npm run build; output directory: build (for CRA) or .next for Next.js.
3. Set public/env vars via Vercel dashboard (REACT_APP_API_URL).

CI/CD snippet (summary): run tests on PR, on merge to main deploy backend and frontend automatically.

11. Performance & Scalability Recommendations

- **Index frequently-queried fields** (skill tags, userId, createdAt).
 - **Cache** popular search results in Redis with TTL (e.g., top skill combinations).
 - **Horizontal scale**: multiple Node instances behind a load balancer; use sticky sessions or Redis adapter for sockets.
 - **Partitioning**: shard large message collections by swapId if needed.
 - **CDN**: host static assets (images, JS/CSS) on CDN via Vercel.
-

12. Project Timeline (Example 6–8 week plan)

- **Week 1**: Requirements + wireframes + basic React pages.
 - **Week 2**: Auth system + user profiles + DB models.
 - **Week 3**: Search & matchmaking core logic + unit tests.
 - **Week 4**: Swap lifecycle + chat integration + message persistence.
 - **Week 5**: Styling, accessibility, responsive design.
 - **Week 6**: Testing, load testing, documentation, and CI.
 - **Week 7**: Deploy to Render/Vercel; address production bugs.
 - **Week 8**: Polish, add gamification, prepare readme and demo video.
-

13. What to Highlight to Recruiters (Talking Points)

- Full-stack ownership: designed and shipped frontend + backend + CI/CD.
 - Real-time system: Socket.io integration and persistence under load.
 - Explainable algorithm: deterministic matchmaking that can be tuned.
 - Production readiness: logging, tests, and deployment to Vercel/Render.
-

14. Conclusion & Future Enhancements

SkillsSwap demonstrates end-to-end systems thinking on a compact, job-ready project. Future improvements: calendar booking, video chat integration, AI skill recommendations, payment gateway (optional), and mobile app.